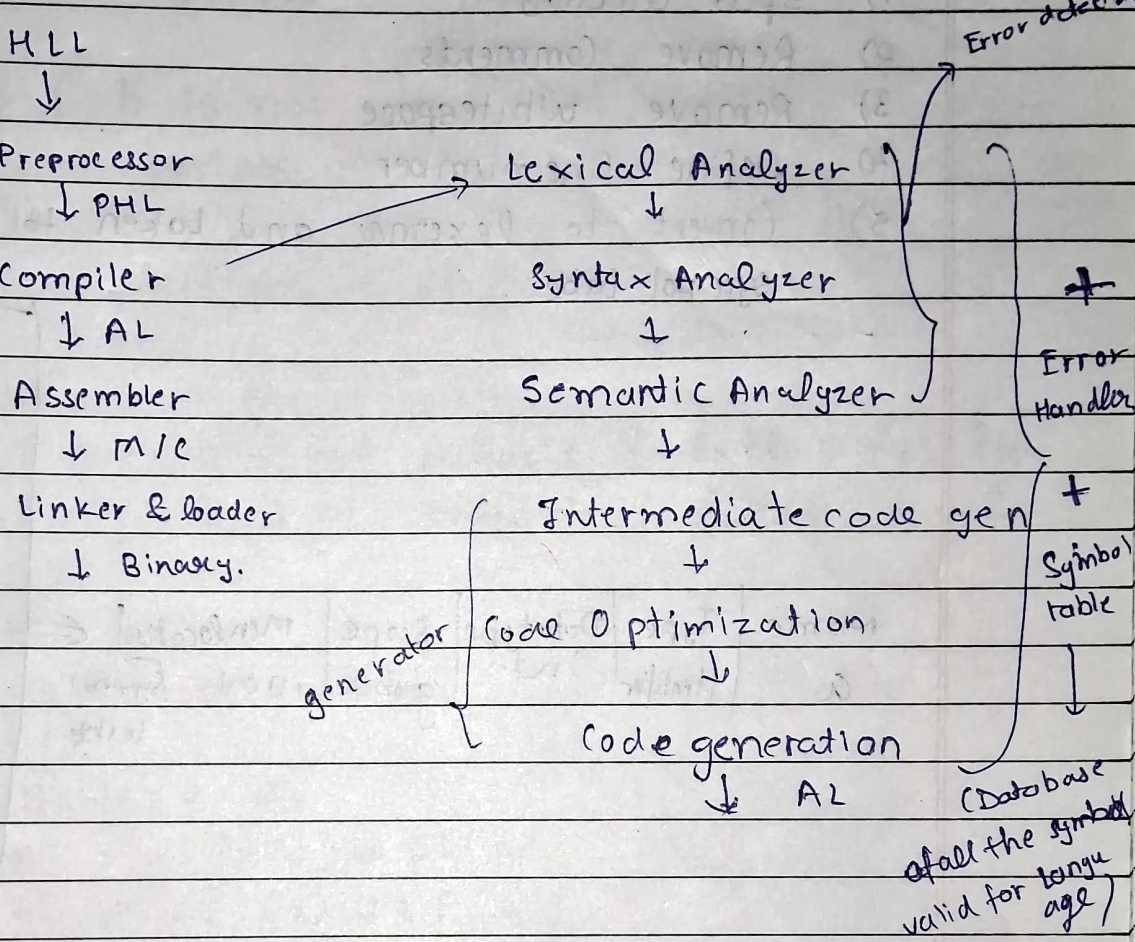
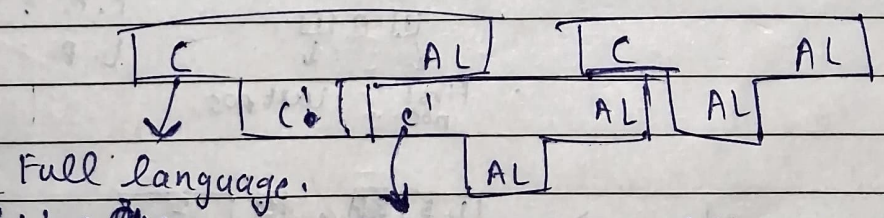
HLL

Unit - I Intro & Lexical Analyzer.



Types of Compiler:

1) Single Pass ⇒ Executed in single pass
2) Multi Pass ⇒



[libs are not AL]

Bootstrapper ⇒ AL code that helps to map the main C code to AL

Lexical Analyser

- Jobs:
  1) Spell checking
  2) Remove Comments
  3) Remove Whitespace
  4) Define line number
  5) Convert to lexemes and token to store in symbol table.

eg:
int a = 10;

lexemes	token
datatype	int
var	a
op	=
value	10
S. op	;

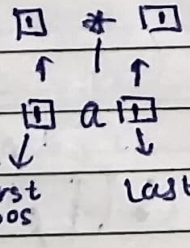
Name	Type	Datatype	Scope	memlocation
a	identifier	int	global	1001

⟸ Symbol table

- R.F to D.F.A.

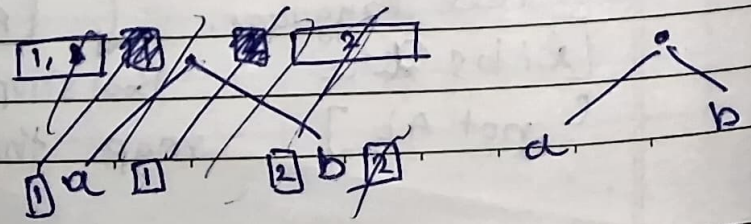
First pos, Last pos, Follow pos and Nullable

⇒ a^* ⇒



Nullable = Yes
F.P. = } Same as
L.P. = } the children

⇒ $a \cdot b$ ⇒



Next node is *

if (a is not nullable)

First pos => fa ∪ fb

else

First pos => fa

if (b is not nullable)

Last pos => la ∪ lb

else

Last pos => lb

Nullable = No

F.P = a is not nullable ? fa : fa ∪ fb

L.P. = b is not nullable ? lb : lb ∪ la

=> a + b or a / b

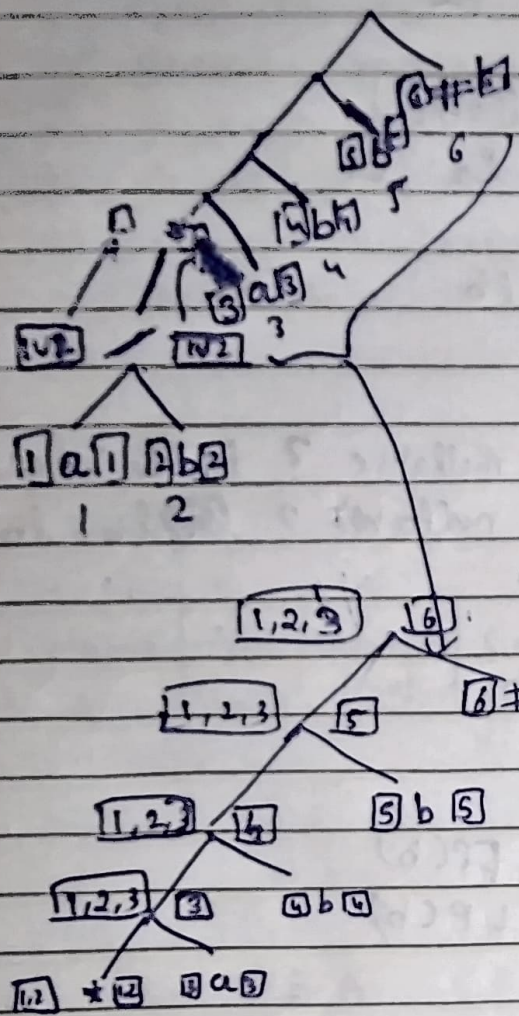
Nullable = No

F.P. = FP(a) ∪ FP(b)

L.P. = LP(a) ∪ LP(b)

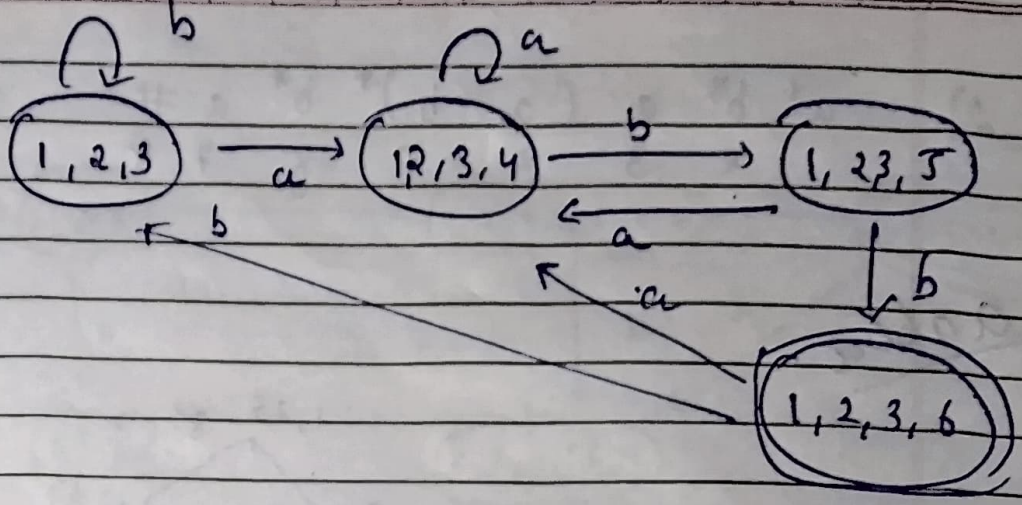
Eg: $(a+b)^* a\ b\ b\ \#$ → end of string

$\quad\quad 1\ 2\ \ 3\ 4\ 5\ 6$



Derivation tree

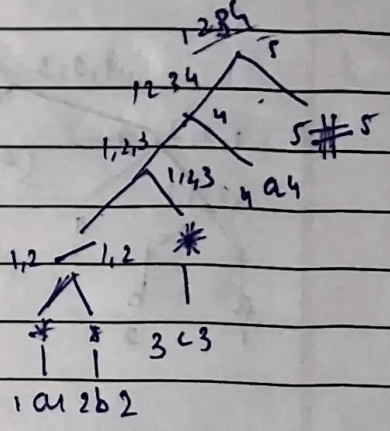
	Node	follow	→ select node then see where is a and b [If two follow position union]
a	1	{1,2,3}	
b	2	{1,2,3}	
a	3	{4}	
b	4	{5}	
b	5	{6}	
#	6	–	

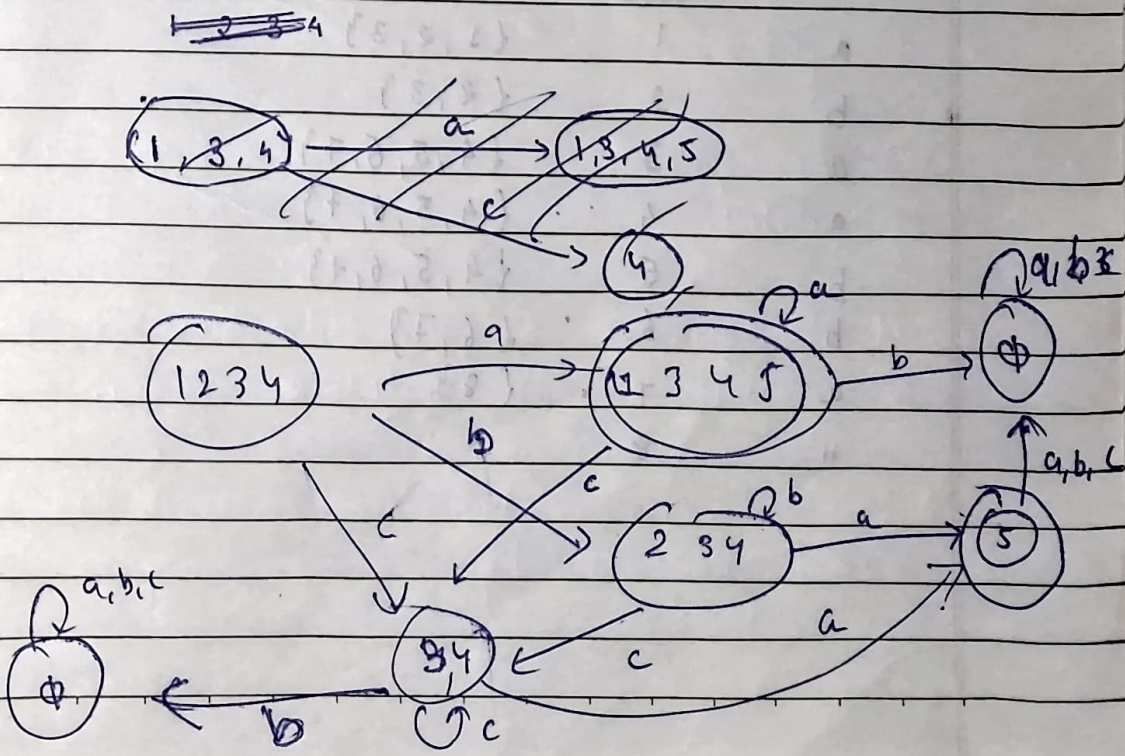


eg: $(a^* + b^*)\, c^*\, a$

1 2 3 4

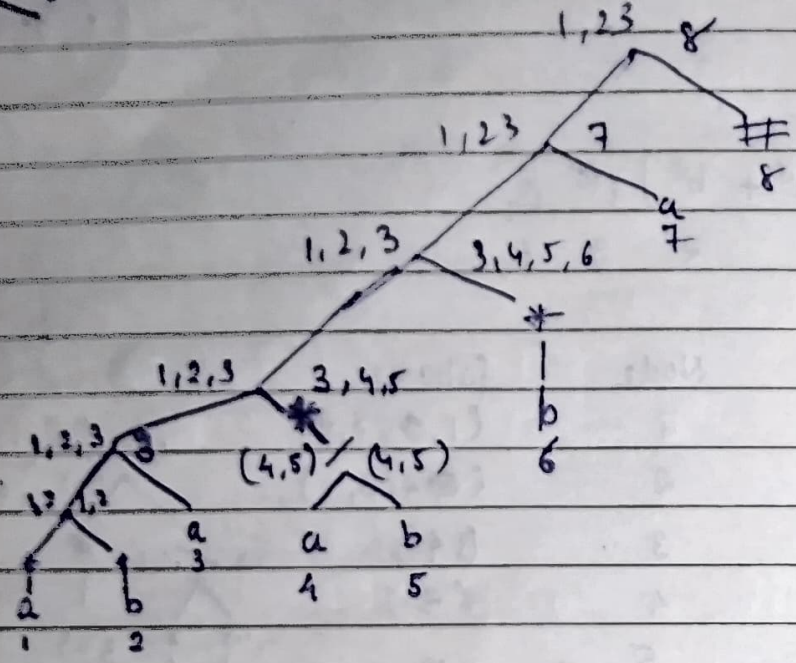
a	Node	follow
a^*	1	{1, 3, 4}
b^*	2	{2, 3, 4}
c^*	3	{3 4}
a	4	{5}
#	5	-





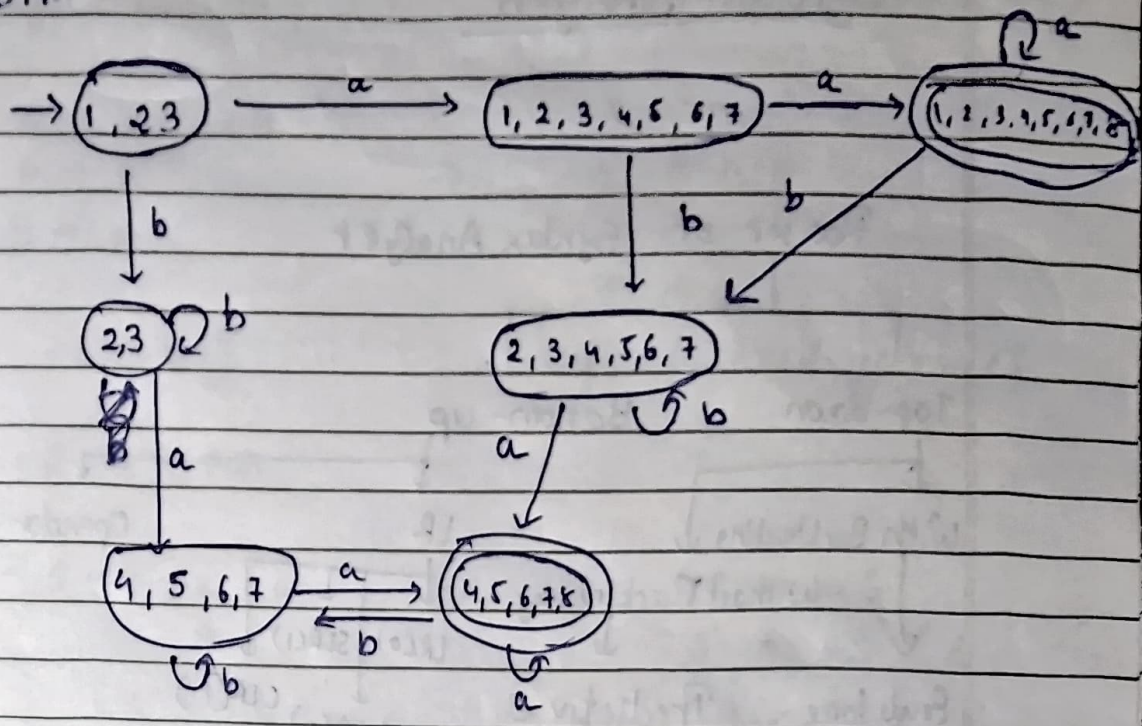
2) a* b* a (a/b)* b* a #
1 2 3 4 5 6 7 8

Solution

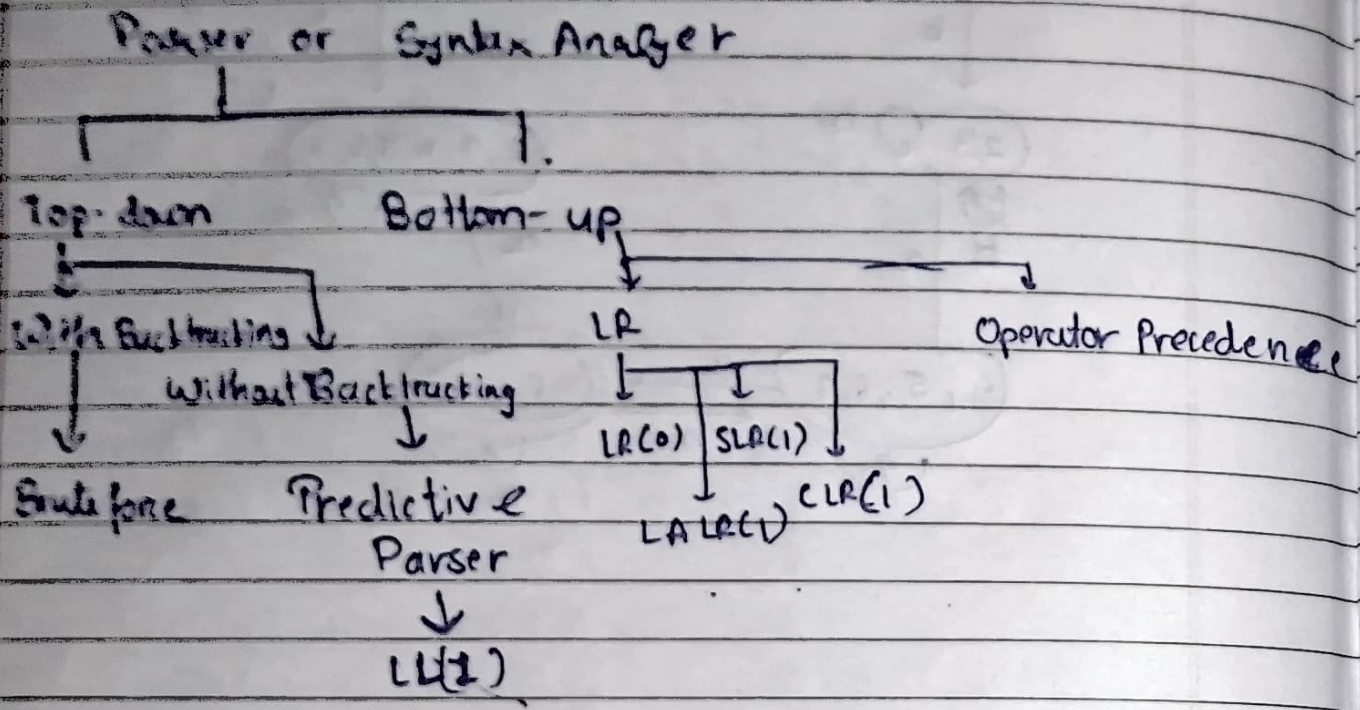


	Node	Follow
a	1	{1, 2, 3}
b	2	{2, 3}
a	3	{4, 5, 6, 7}
a	4	{4, 5, 6, 7}
b	5	{4, 5, 6, 7}
b	6	{6, 7}
a	7	{8}
#	8	-

DFA:



Syntax Analyser



- ~~Top-Down Parser~~

- For CFG,

→ If the grammer is ambigueous then, parser can't be created.

→ If the grammer is Non-dertminstric then, parser can't be created except Brute force parser. or convert in Deterministic

→ If the grammar has left recursion then also parser can't be created.

Ambiguous Grammar :- More than one derivation tree possible

- Left - factoring (Non- Deterministic → Deterministic)

$S \to aA / aB$
$A \to a$
$B \to a$

$\Rightarrow$

$S \to aS'$
$S' \to A/B$
$A \to a$
$B \to B$

Left - factored grammer

- $LR \to RR$

$S \to Sa / a \to$

$S \to aS'$
$S' \to aS' / \epsilon$

eg: $S \to Sa / Sb / c$

$S \to Sx / c$
$x \to a/b$

$S \to cS'$

$S' \to aS' / bS' / \epsilon$

$S \to cS'$
$S' \to xS' / \epsilon$
$x \to a/b$

eg $S \to Sa / Sb / x / y$

$S \to Sz / x / y$
$z \to a/b$

$S \to xS' / yS'$
$S' \to aS' / bS' / \epsilon$

$S \to xS' / yS'$
$S' \to zS' / \epsilon$
$z \to a/b$

eg: $S \to Sa \mid b \mid c$

$S \to bS' \mid cS'$

$S' \to aS' \mid \epsilon$

Brute force (With Backtracking)

- Brute force will work for non-deterministic grammar unlike other parser methods.

- eg:
$S \to aAB \mid aXYb \mid aB$

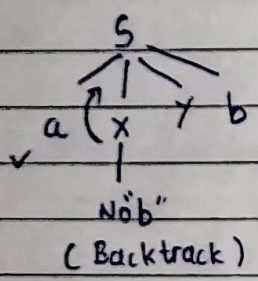
$A \to bB \mid bA \mid \epsilon$

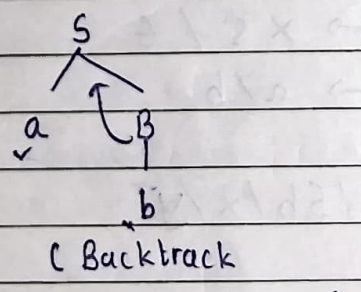
$B \to b$

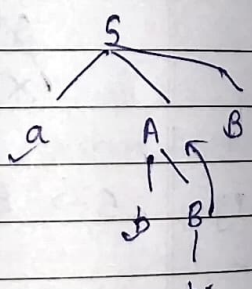
$X \to a$

$Y \to d$

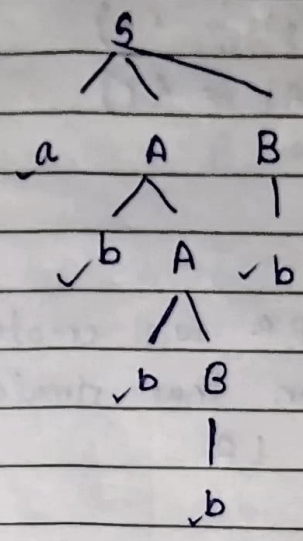
Derivation tree $\quad w = abbbb$







ϵ : like a pseudo terminal it is actual terminal if there is no other option.
And also it counted towards "first"



This occur after multiple backtracking. (Thus, costly)

* Without Backing → Predictive Parser → LL(1)

for each production

- First : first occuring terminal in derivation tree

eg:-

$S \to a\ A\ B \Rightarrow f(S) = a$

$A \to b / c \Rightarrow f(A) = \{b, c\}$

$B \to d / e \Rightarrow f(B) = \{d, e\}$

$S \to A\ B \Rightarrow f(S) = \{a, \epsilon, b\}$

$A \to a / \epsilon \Rightarrow f(A) = \{a, \epsilon\}$

$B \to b / \epsilon \Rightarrow f(B) = \{b, \epsilon\}$

$b \in$ A,B $\Rightarrow \epsilon$

If $S \to A\ B\ d \Rightarrow f(s) = \{a, \epsilon, b, d\}$

$b \in A \Rightarrow \epsilon$

If $S \to A\ B\ d\ X \Rightarrow f(s) = \{a, \epsilon, b, d\}$

$X \to x$

Q: $S \to SaA / b$ ⇒ $f(S) = \{b\}$

$A \to Ab / c$ ⇒ $f(A) = \{c\}$

↓

$S \to bS'$

$S' \to aAS' / \epsilon$

$A \to cA'$

$A' \to bA / \epsilon$

Convert RR will create confusion thus simulate the tree for LR

eg: $S \to AaBb / BbAa$ ⇒ $f(S) = \{\epsilon, a, b\}$

$A \to \epsilon$ ⇒ $f(A) = \{\epsilon\}$

$B \to \epsilon$ ⇒ $f(B) = \{\epsilon\}$

eg: $E \to E+T / T$ ⇒ $f(E) = \{(, id\}$

$T \to T*F / F$ ⇒ $f(T) = \{(, id\}$

$F \to (E) / \underbrace{id}_{\text{id is together}}$ ⇒ $f(F) = \{id, (\}$

- Follow: if $A \to \alpha A\beta$, $\beta \neq$ null

$fol(A) = first(\beta)$

if $A \to \alpha\beta$

$fol(\beta) = fol(A)$

if $S \to \alpha A\beta$, β = null

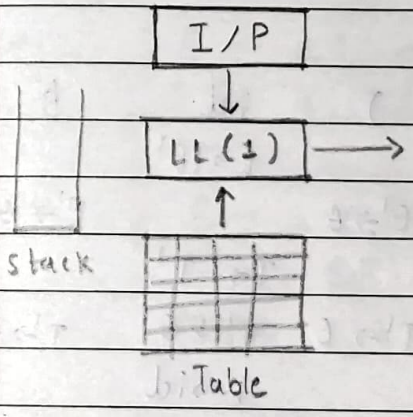
$fol(A) = f(\beta) \cup fol(S) - \epsilon$

[For Start production if no follow can found use '$']

eg: $S \rightarrow aSA \Rightarrow f(S) = \{a\}$
$A \rightarrow bB \Rightarrow f(A) = \{b\}$
$B \rightarrow \epsilon \Rightarrow f(B) = \{a\}$

$fol(S) = first(A) = \{\$, a\}$
$fol(A) = first(\epsilon) \cup fol(S) - \epsilon$
$= fol(S)$
$= \{\$, a\}$ same like
$fol(B) = fol(A) = \{\$, a\}$

- LL(1) → 1-Look ahead Symbol
  - L → Left Recursion
  - Left to Right Screening



eg: $E \rightarrow E+T / T$
$T \rightarrow T*F / F$
$F \rightarrow (E) / id$

Removing L.R. : $E \rightarrow TE' \quad E' \rightarrow +TE' / \epsilon$
$T \rightarrow FT' \quad T' \rightarrow *FT' / \epsilon$
$F \rightarrow (E) / id$

Creating Table:

	+	*	(	)	id	$
E						
E'						
T						
T'						
F						

First :

$E \Rightarrow \{(, id\}$

$E' \Rightarrow \{+, \epsilon\}$

$T \Rightarrow \{(, id\}$

$T' \Rightarrow \{*, \epsilon\}$

$F \Rightarrow \{(, id\}$

Follow :

$E \Rightarrow \{\$,)\}$

$E' \Rightarrow \{\$,)\}$

$T \Rightarrow \{+, \$,)\}$

$T' \Rightarrow \{+, \$,)\}$

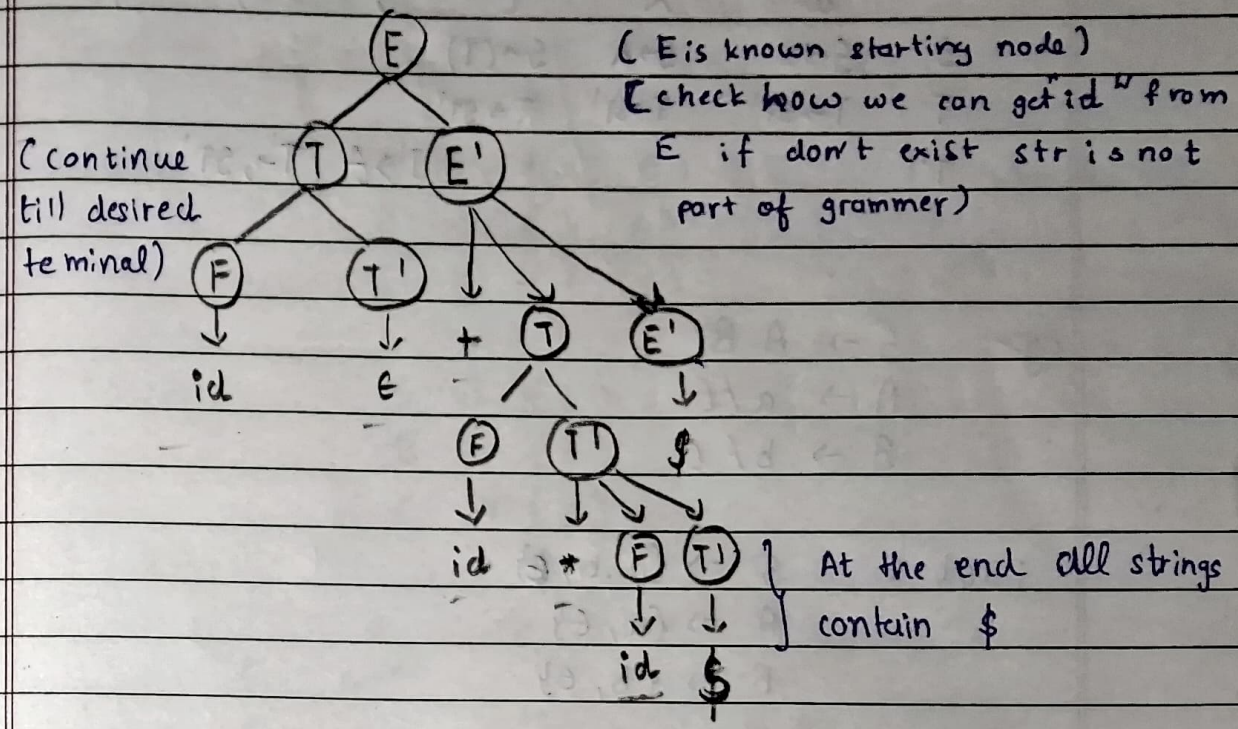
$F \Rightarrow \{*, +, \$,)\}$

Updating table:

	+	*	(	)	id	$
E			$E \to TE'$		$E \to TE'$	
E'	$E' \to +TE'$			$E' \to \epsilon$		$E' \to \epsilon$
T			$T \to FT'$		$T \to FT'$	
T'	$T' \to \epsilon$	$T' \to *FT'$	~~$T' \to *FT'$~~	$T' \to \epsilon$	~~$T' \to \epsilon$~~	$T' \to \epsilon$
F			$F \to (E)$		$F \to id$	

* If, there are more than one production in one cell then LL(1) Parser for the grammer will not exist
⇒ This will generally won't ~~if~~ occur if already checked that the grammer is determinitic and non-abigueous

eg: I/P : id + id * id $



eg: S → a / ∧ / (T)
T → T, S / S

⇒ S → a / ∧ / (T)
T → ST'
T' → , ST' / ε

First : S ⇒ {a, ∧, (}
T ⇒ {a, ∧, (}
T' ⇒ {, , ε}

Follow : S ⇒ ~~{$, , a, ∧, (}~~ {$, , ,)}
T ⇒ {)}
T' ⇒ {)}

Table:

	a	^	(	)	,	$
S	S→a	S→^	S→(T)			
T	T→ST'	T→ST'	T→ST'			
T'				T'→ε	T'→,ST'	

eg: S → A B
A → a/ε
B → b/ε

First : S => {a, b, ε}
A => {a, ε}
B => {b, ε}

Follow: S => {$}
A => {b, $}
B => {$}

	a	b	$
S	S→AB	S→AB	S→AB
A	A→a	A→ε	A→ε
B		B→b	B→ε

eg $S \to aB / \epsilon$
$B \to bC / \epsilon$
$C \to cS / \epsilon$

First : $S \Rightarrow \{a, \epsilon\}$
$B \Rightarrow \{b, \epsilon\}$
$C \Rightarrow \{c, \epsilon\}$

Follow : $S \Rightarrow \{\$\}$
$B \Rightarrow \{\$\}$
$C \Rightarrow \{\$\}$

	a	b	c	$
S	$S \to aB$			$S \to \epsilon$
B		$B \to bC$		$B \to \epsilon$
C			$C \to cS$	$C \to \epsilon$

eg: $S \to aSbS / bSaS / \epsilon$

First : $S \Rightarrow \{a, b, \epsilon\}$ Ambig
Follow : $S \Rightarrow \{\$, a, b\}$

	a	b	$
S	$S \to aSbS$	$S \to bSaS$	$S \to \epsilon$

eg: S → aAa / ε
A → abS / ε

First : S => {a, ε}
A => {a, ε}

Follow : S => {$, a}
A => {a}

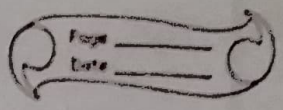
	a	b	$
S	S→aAa, S→ε		S→ε
A			

eg S → aABb
A → c / ε
B → d / ε

First: S => {a}
A => {c, ε}
B => {d, ε}

Follow: S => {$}
A => {d, b}
B => {b}

	a	b	c	d	$
S	S→aABb				
A		A→ε	A→c	A→ε	
B		B→ε		B→d	



e.g. $E \rightarrow E+i/i$

$\Downarrow$

$E \rightarrow iE'$

$E' \rightarrow +iE'/\epsilon$

```
main() {
    E();
    if (l == '$')
        printf("Prasing");
}

E() {
    if (l == 'i') {
        match('i');
        E'();
    }
}

E'() {
    if (l == '+') {
        match('+');
        match('i');
        E'();
    } else {
        return;
    }
}

match(char t) {
    if (l == t) {
        l = getchar();
    else {
        printf("error");
    }
}
```

* L R (1) → look ahead

L → Left to right scan

R → Reverse of rightmost derivation

eg: S → AaAb ① / BbBa ②

B → b ③

A → a ④

→ We augment because there may recursion S to know that it end of recursion we create this production

S' → .S

S → .AaAb

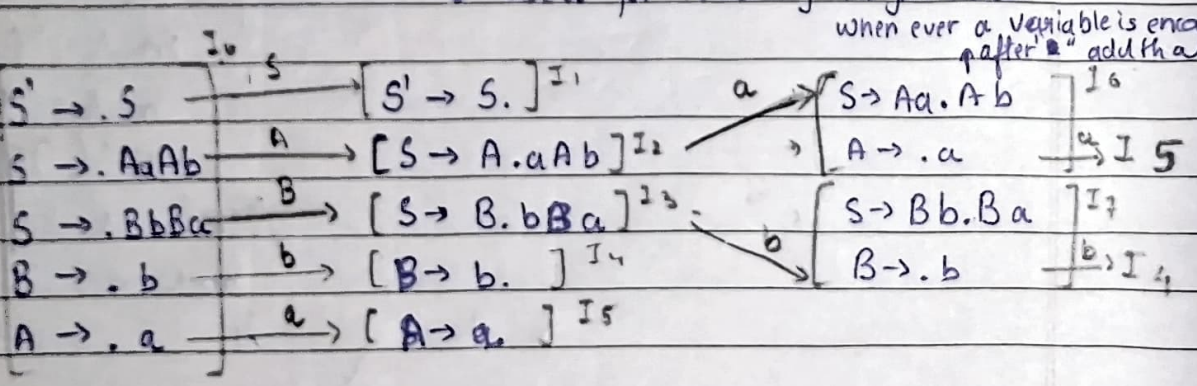
S → .BbBa

B → .b

A → .a

↳ "." used for tracking in grammar.

when ever a variable is encountered after "." add that



[S → Aa.Ab] I6 —A→ [S → AaA.b] I8 —b→ [S → AaAb.] I10

[S → Bb.Ba] I7 —B→ [S → BbB.a] I9 —a→ [S → BbBa.] I11

	Action			Go-to		
	a	b	$	S	A	B
I_0	S_5	S_4		1	2	3
I_1			accept			
I_2	S_6					
I_3		S_7				
I_4	R_3	R_3	R_3			
I_5	R_4	R_4	R_4			
I_6	S_5				8	
I_7		S_4				9
I_8		S_{10}				
I_9	S_{11}					
I_{10}	R_1	R_1	R_1			
I_{11}	R_2	R_2	R_2			

reduce 3rd prod. B → b

ered prod

=> str : a a a b $

Stack : 0 a 5

[I_0][a]

↓ check what is I_5 [It is R_4]

0 A 2 [Pop double of the total elements of Prod.]

[I_0][A]

↓ I_2

0 A 2 a 5

↓ After Reduction see [I_6][A]

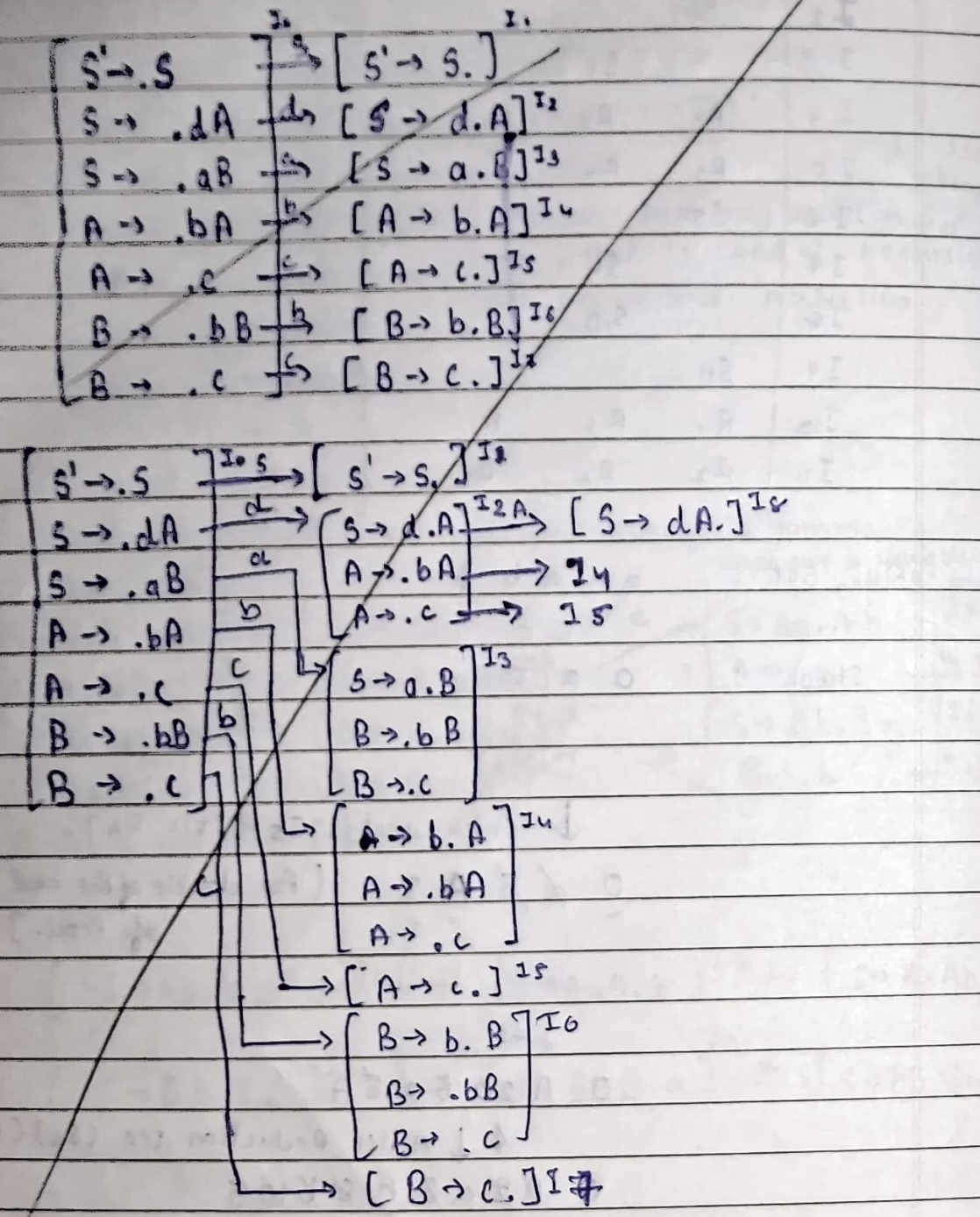
0 A 2 A 8

↓ [I_0][S]

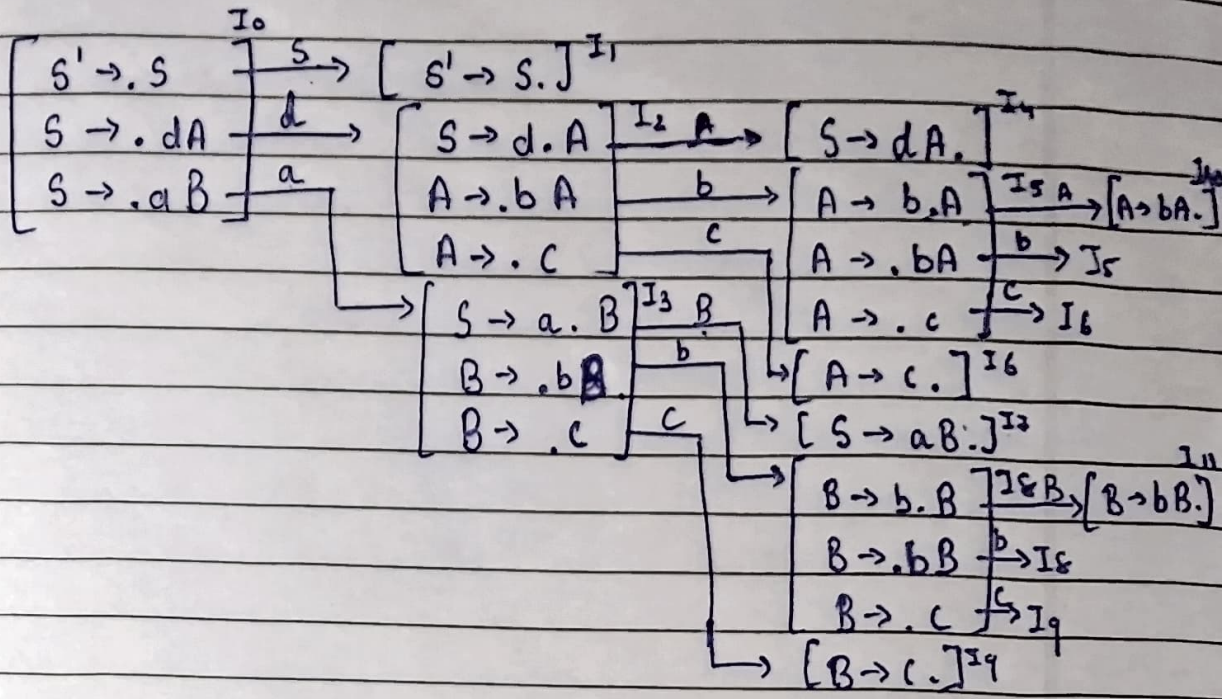
0 S 1

Accept

3. $S \to dA / aB$
$A \to bA / c$
$B \to bB / c$



eg: $S \to dA \,(1) \;/\; aB \,(2)$
$A \to bA \,(3) \;/\; c \,(4)$
$B \to bB \,(5) \;/\; c \,(6)$



	Actions					GO-TO		
	a	b	c	d	$	S	A	B
I_0	S_3		S_2			1		
I_1					Accept			
I_2		S_5	S_6				4	
I_3		S_8	S_9					7
I_4	R_1	R_1	R_1	R_1	R_1			
I_5							10	
I_6	R_4	R_4	R_4	R_4	R_4			
I_7	R_2	R_2	R_2	R_2	R_2			
I_8								11
I_9	R_6	R_6	R_6	R_6	R_6			
I_{10}	R_3	R_3	R_3	R_3	R_3			
I_{11}	R_5	R_5	R_5	R_5	R_5			